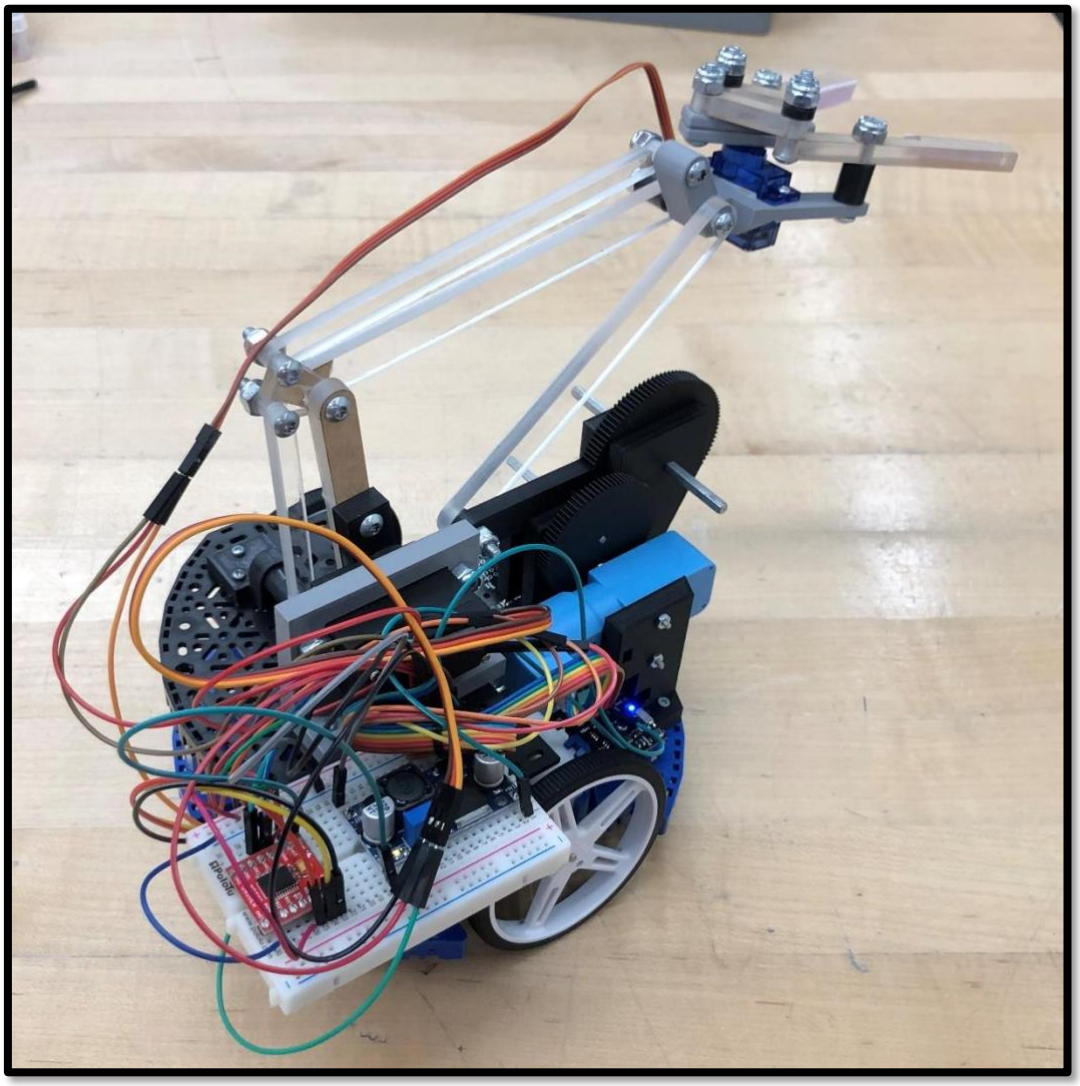
Final Report:
Romi 5 Bar System

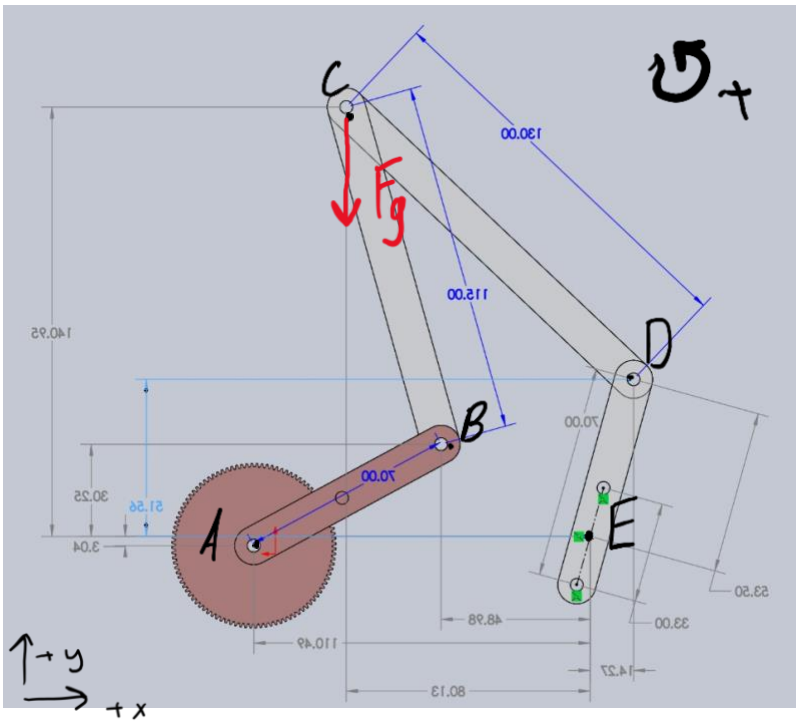
Team 6



Mark Kheyfets

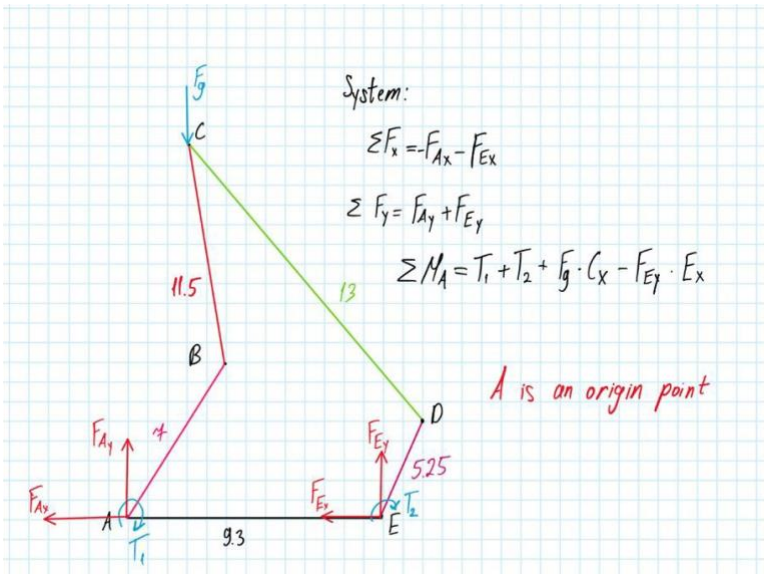
1. Free Body Diagram analysis

Five Bar (dimensions are in mm):

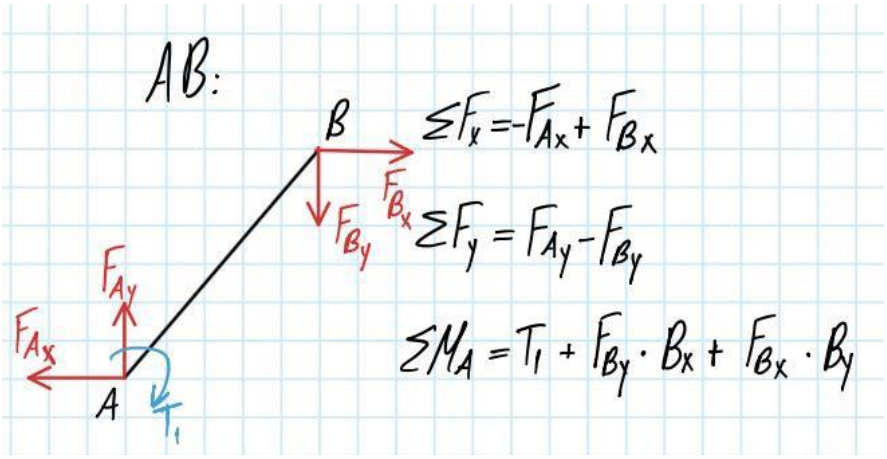


Gripper Mass is 70.67 g so $F_{gripper}$= 0.6933 N

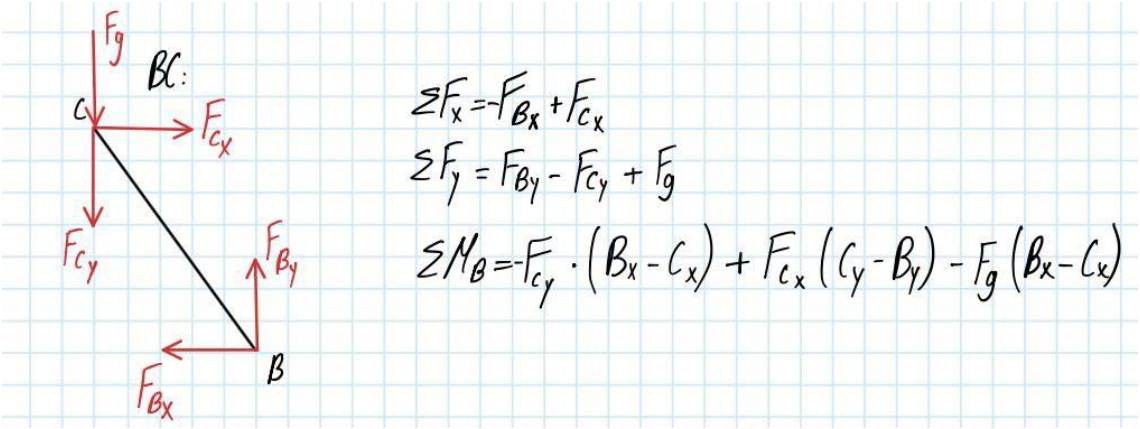
System:



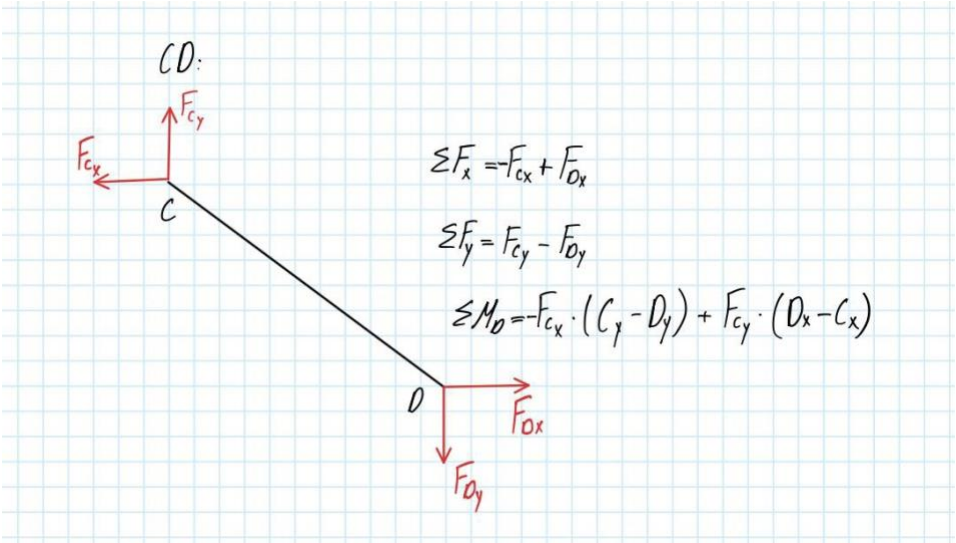
Link AB:



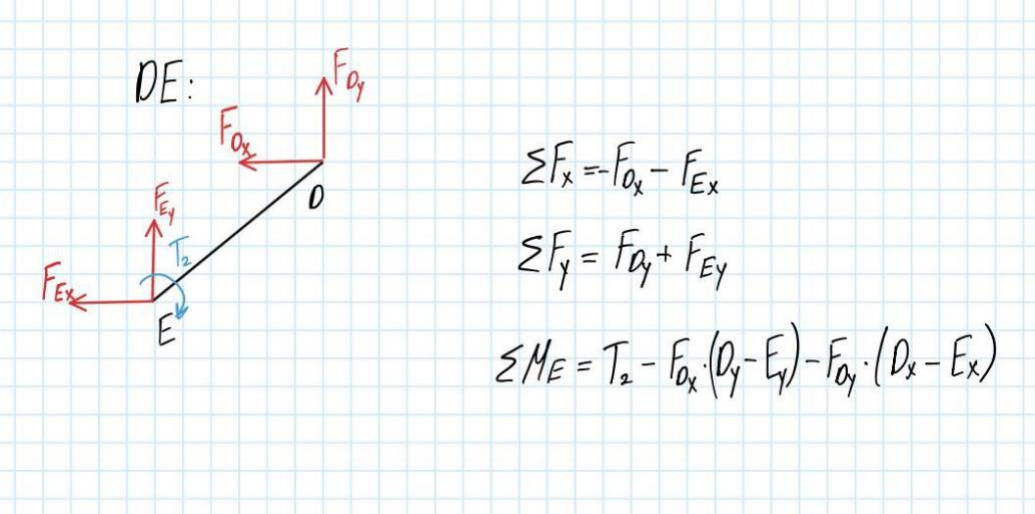
Link BC:



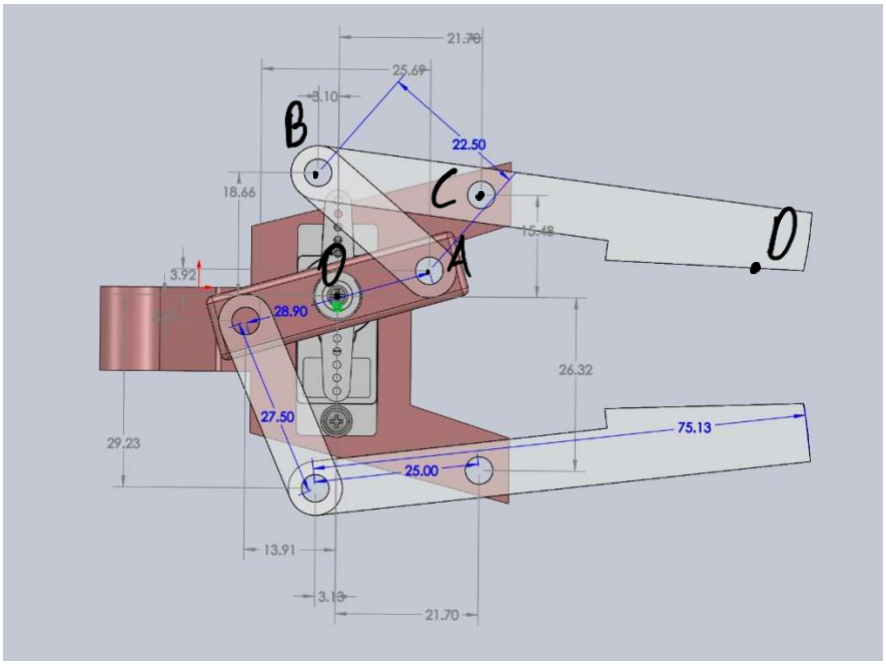
Link CD:



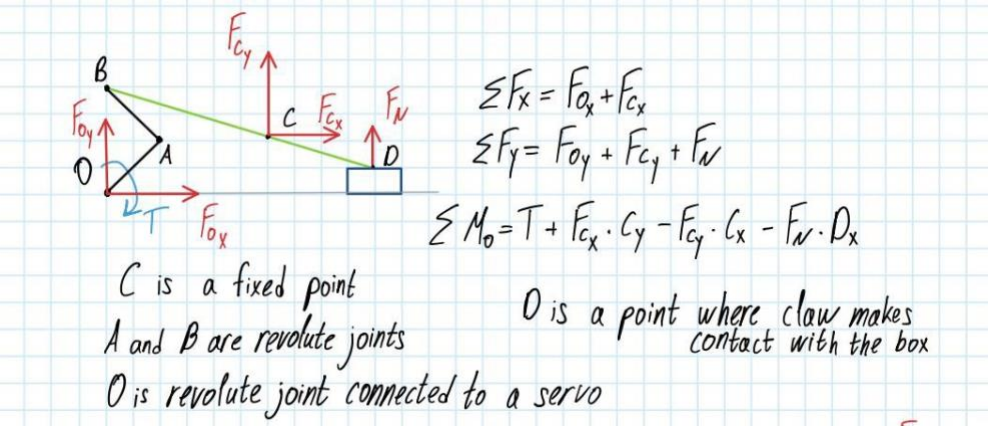
Link DE:



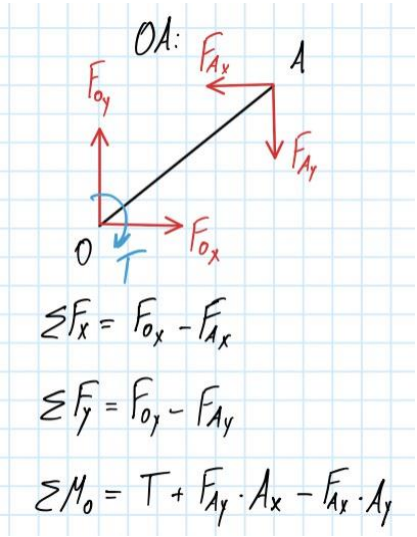
Gripper (dimensions are in mm):



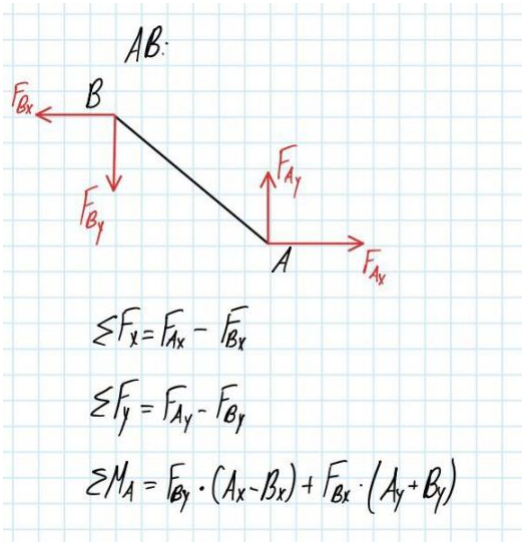
System:



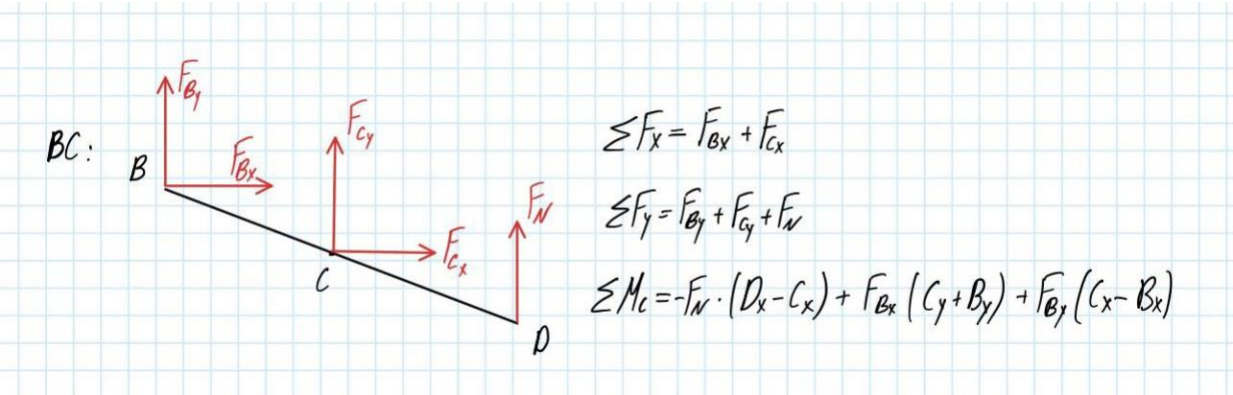
OA:



AB:



BC:



2. MATLAB calculation of motor torques

at a five-bar with a vertical force of 5N as the only force at the joint (MATLAB code and print-out attached to report). (7 points)

Note: The measurements for the MATLAB code were taken while the 5-bar mechanism was in a different position than shown in the SolidWorks screenshot in Part 1. As a result, the coordinates of the points may vary slightly.

Code:

```
clear; clc; close all;

AB = 7;    % cm
BC = 11.5;    % cm
CD = 13;    % cm
DE = 5.25;    % cm
AE = 9.3;    % cm
F  = 5;     % N

Ax = 0;
Ay = 0;
Bx = 4.87;
By = 5.03;
Cx = 5.74;
Cy = 16.49;
Dx = 12.4;
Dy = 5.33;
Ex = 9.26;
Ey = 0.63;

syms FAx FAy FBx FBy FCx FCy FDx FDy FEx FEy T1 T2


% AB equilibrium equations:
eq1 = - FAx + FBx == 0; %sum of Fx
eq2 = FAy - FBy == 0; %sum of Fy
eq3 = T1 + FBy * Bx + FBx * By == 0; %sum of moments about A

% BC equilibrium equations:
eq4 = - FBx + FCx == 0; %sum of Fx
eq5 = FBy - FCy - F == 0; %sum of Fy
eq6 = FCy * (Bx - Cx) - F * (Bx - Cx) + FCx * (Cy - By) == 0; %sum of moments about B

% CD equilibrium equations:
eq7 = - FCx + FDx == 0; %sum of Fx
eq8 = FCy - FDy == 0; %sum of Fy
eq9 = -FCx*(Cy-Dy)+FCy*(Dx+Cx) == 0; %sum of moments about

% DE equilibrium equations:
eq10 = - FEx - FDx == 0; %sum of Fx
eq11 = FEy + FDy == 0; %sum of Fy
eq12 = T2 - FDy * (Dx - Ex) - FDx * (Dy - Ey) == 0; % Moment around Point E
```

```
% System equations:
eq13 = - FAx - FEx - F == 0; %sum of Fx
eq14 = FAy + FEy == 0; %sum of Fy
eq15 = T1 + T2 - (F * Cx) - (FEy * Ex) == 0; % Moment around Point A

equations = [eq1, eq2, eq3, eq4, eq5, eq6, eq7, eq8, eq9, eq10, eq11, eq12];

solution = solve(equations, [FAx, FAy, FBx, FBy, FCx, FCy, FDx, FDy, ...
    FEx, FEy, T1, T2], 'Real', true);

% Extract numerical values
FAx_sol = double(solution.FAx);
FAy_sol = double(solution.FBy);
FBx_sol = double(solution.FBx);
FBy_sol = double(solution.FBy);
FCx_sol = double(solution.FCx);
FCy_sol = double(solution.FCy);
FDx_sol = double(solution.FDx);
FDy_sol = double(solution.FDy);
FEx_sol = double(solution.FEx);
FEy_sol = double(solution.FEy);
T1_sol = double(solution.T1);
T2_sol = double(solution.T2);


T1_sol_Ncm = T1_sol * 100;  % from N·m to N·cm
T2_sol_Ncm = T2_sol * 100;  % from N·m to N·cm


fprintf(' FAx = %8.3f N\n',    FAx_sol);
fprintf(' FAy = %8.3f N\n',    FAy_sol);
fprintf(' FBx = %8.3f N\n',    FBx_sol);
fprintf(' FBy = %8.3f N\n',    FBy_sol);
fprintf(' FCx = %8.3f N\n',    FCx_sol);
fprintf(' FCy = %8.3f N\n',    FCy_sol);
fprintf(' FDx = %8.3f N\n',    FDx_sol);
fprintf(' FDy = %8.3f N\n',    FDy_sol);
fprintf(' FEx = %8.3f N\n',    FEx_sol);
fprintf(' FEy = %8.3f N\n',    FEy_sol);
fprintf(' T1 = %8.3f N·m  (%.3f N·cm)\n', T1_sol, T1_sol_Ncm);
fprintf(' T2 = %8.3f N·m  (%.3f N·cm)\n', T2_sol, T2_sol_Ncm);
```

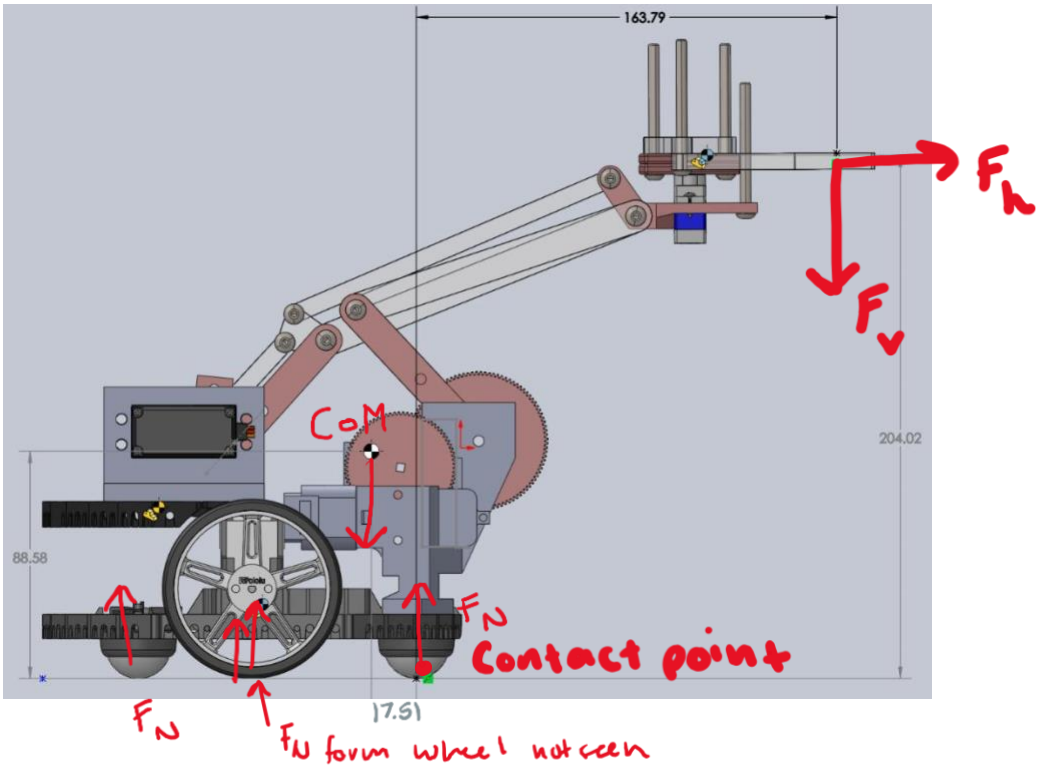
Print-out:

```
FAx =   -0.398 N
FAy =    4.755 N
FBx =   -0.398 N
FBy =    4.755 N
FCx =   -0.398 N
FCy =   -0.245 N
FDx =   -0.398 N
FDy =   -0.245 N
FEx =    0.398 N
FEy =    0.245 N
T1 =  -21.154 N·m  (-2115.418 N·cm)
T2 =   -2.641 N·m  (-264.063 N·cm)
```

3. Calculation of the maximum end-effector forces

To find the tipping point, we first set our gripper out as far as possible. This shows the "worst case scenario". This is because when the gripper is as far out as possible, the Center of Mass (CoM) of the assembly is the closest already to tipping due to the weight of the gripper. We then used SolidWorks to find the Center of Mass of the entire assembly. To do this, we first had to define as many materials as we could. The center of mass is the white and black dot. We also need to know the dimensions of the CoM and Gripper in comparison to the farthest contact point of the Romi and the ground. With this info, we need to calculate the force needed to move the center of gravity past this contact point.

Photos of Dimensions and Mass Properties of Assembly (in mm):



Mass Properties of the Assembly:

```
Mass properties of help
   Configuration: Default
   Coordinate system: -- default --

Mass = 572.38 grams

Volume = 447983.79 cubic millimeters

Surface area = 274038.22  square millimeters

Center of mass: ( millimeters )
     X = -18.35
     Y = -1.26
     Z = -34.72
```

Fg = 572.38g /1000 * 9.81 m/s^2 = 5.62 N

When the robot is tipping, all normal forces FN must be zero, except for the normal force at the contact point (the right ball). This is because if the robot is about to tip over, there will be no normal forces on the back ball and wheels as these parts will be lifted off the ground. When these normal forces are zero and the robot is about to tip, the CoM is vertically above the right ball. If the force we greater than zero, there would still be contact, and it is physically impossible for them to be zero.

Solve for vertical force (no horizontal):

Summing moments about the pivot contact point, we get:

$\sum M_{pivot}$ = Fg (17.51 mm) - Fv(163.79 mm) = 0 ➔ 5.62 N(17.51 mm) = Fv(163.79 mm) ➔ Fv = **0.60 N**

This is the force to be on the verge of tipping. Any force larger than this would tip Romi.

While this seems low, due the weight of the gripper and the assembly being set to as far as possible, it would make sense that this "extra" load would be on the smaller side.

Solve for horizontal force (no vertical):

Summing moments about the pivot contact point, we get:

$\sum M_{pivot}$= Fg (17.51 mm) - Fh(204.02 mm) = 0 ➔ 5.62 N (17.51 mm) = Fv(204.02 mm) ➔ Fh = **0.48 N**

Again, this is the force to be on the verge of tipping, so any force larger than this would cause it to tip.

In context of the lab loads (boxes):

Each load weighs approximately 3 grams = 0.029 N. This would be a downwards force. Therefore, we could theoretically lift 0.6/0.029 = 20 loads in our assembly's most unstable position

Tested:

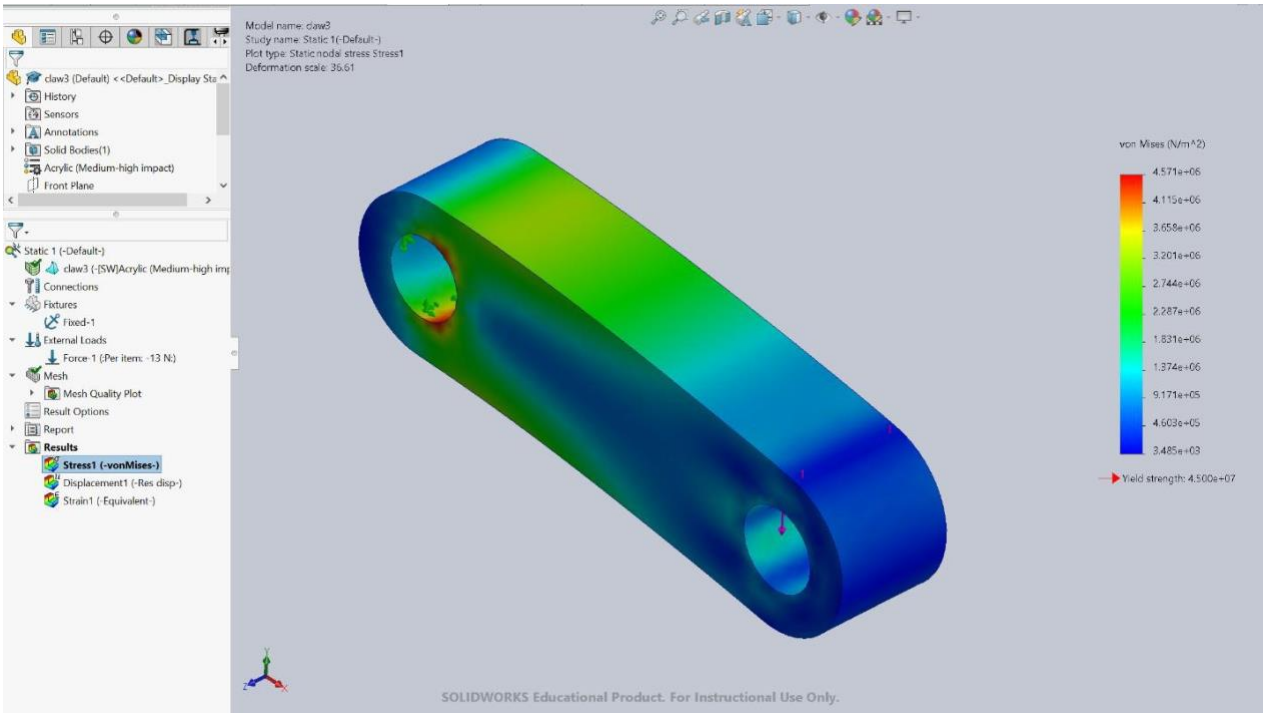
The tested vertical force needed to tip the robot was about 1.1 N. This is larger than our theoretical. This is most likely due to weights not calculated in SolidWorks CoM. This includes motor weights, screws, nuts, and all the wiring. It can also be due to incorrect material properties.

The tested horizontal force is unknown because the robot "slipped" before it tipped. This means that the force to overcome the force of friction between the wheels and the ground is less than the tipping force required. This slip was only on one wheel, meaning our robot's CoM is off to one side. Therefore, one wheel would receive a great downwards force, generating more friction.

4. FEM analysis

We chose to do as much of our project as we could in acrylic. This is mainly because of the is coefficient of friction in comparison to PLA and how quick it is to cut our pieces, allowing us to redesign and assemble faster.

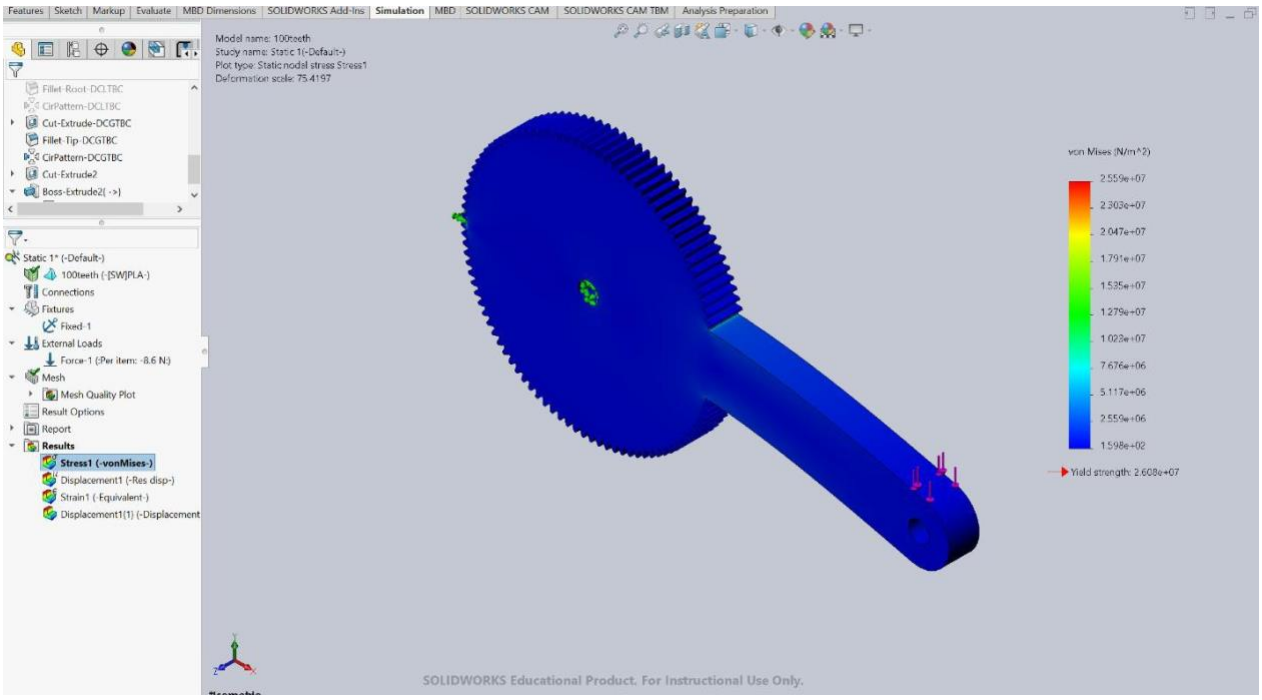
For the gripper, the weakest link is the longer intermediate claw piece. This is because it has the thinnest walls around the attachment points, making them more likely to break, and the longest distance (therefore longest torque/moment arm) from hole to hole.

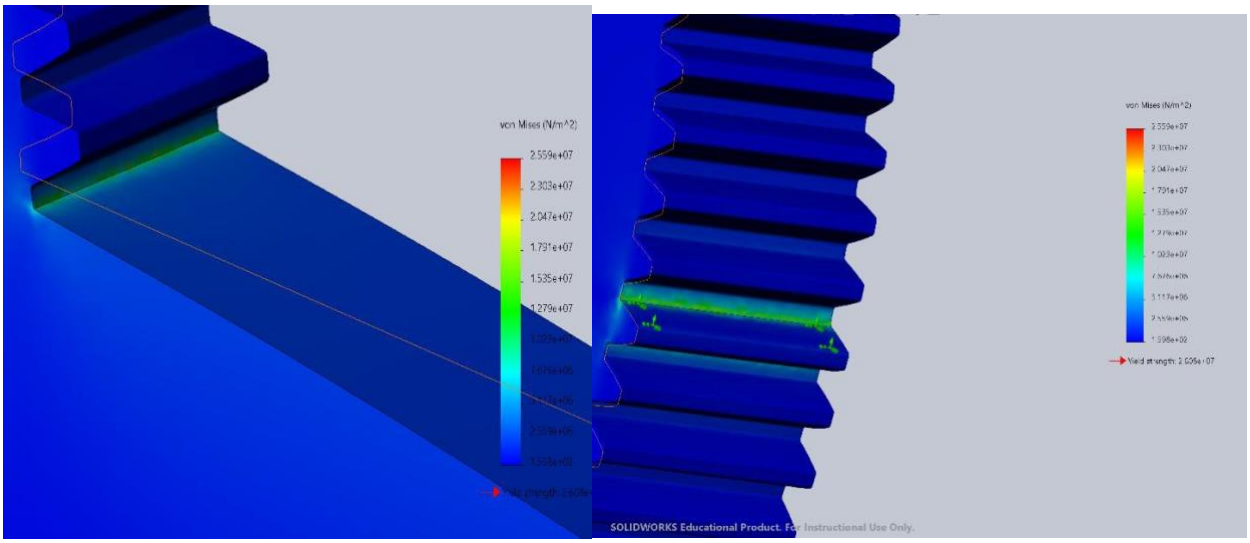


Since we used the premade SolidWorks acrylic material option, the yield strength was 4.5*10^7. With one hole constrained, the max force (mac force before permanent deformation) that can be

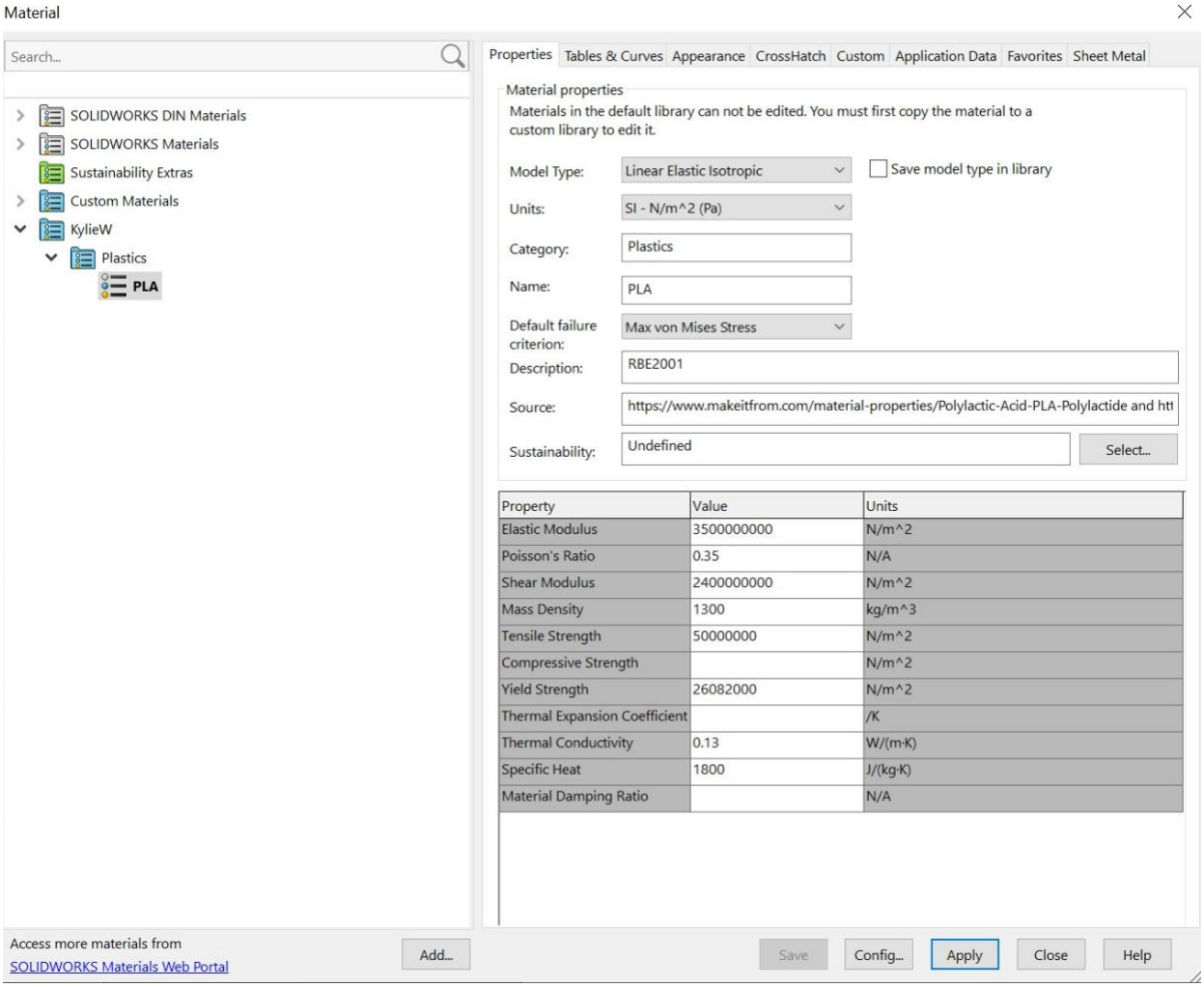
applied to the other hole is 13 N. As one can see, the area with the most stress is the thinnest sections experiencing the most bending.

For the weakest part of the 5-bar, we choose the one link made from PLA and with a gear. This is because it is made of weaker material that is more likely to bend (it has a lower yield strength). It is most likely to break due to how thin the gear teeth are (where high amounts of the torque are being applied) and due to the abrupt connection of the beam to the gear.





For this part, we created a custom PLA material with these specifications:



These specifications were collected from the following websites:

https://www.makeitfrom.com/material-properties/Polylactic-Acid-PLA-Polylactide

https://iopscience.iop.org/article/10.1088/1742-6596/2468/1/012094/pdf#:~:text=Poisson's%20ratio%20of%20PLA%20material%20is%20considered%20to%20be%200.35.

https://pubs.aip.org/aip/acp/article-abstract/2059/1/020038/790230/Preliminary-investigations-of-polylactic-acid-PLA?redirectedFrom=PDF

Thus, the yield strength of this material is 2.6082 * 10^7. The max force of 8.5 at the end connector of the link can be applied before we reach that yield strength. We fixed the opposite tooth and the axle hole to simulate the torque from the 20-tooth gear that would be turning this gear and the axle holding it up. As a result, the breaking point is most likely to be at the teeth. As shown in the photos, that is where the highest stress is.

5. Code

```
#include <Arduino.h>
#include <Romi32U4.h>
#include <Chassis.h>
#include <servo32u4.h>
#include <BlueMotor.h>

//chassis
Chassis chassis;

// Blue motor control variables
BlueMotor blueMotor; // The Blue Motor object

// Servo control variables
Servo32U4Pin5 servo1;
Servo32U4Pin6 servo2;

//For Romi position control
double speed = 20; // set speed to move the romi in cm/sec
double timeToRun = 0.0;
unsigned long startTime = 0;
bool isMoving = false;
bool CW =false;
bool CCW = false;
bool Backwards = false;

//5 bar constants in mm
//B is the black servo and is (0,0)
//A is the blue motor and is at (92.6 , - 6.3)
//P is where the claw is

double BC = 53.5;
double CP = 130;
double DP = 115;
double AD = 70;
double Ay = -6.3;
double Ax = 92.6;
double BP; //going to find these values
double AP;
double gamma;
double delta;
double beta;
double alpha;
double thetaB; //theta of the servo
```

```
double  thetaA;
int blackServoPWM;
int xchange = 10; //how to increment up and down in mm
int ychange = 10;
double Px = 130; //found starting position (black servo at 90 degrees and blue
motor at 0 )
                 // theta of blue motor
double Py = 50; // in mm

//for blue motor;
double ticksPerRev = 280.0; //https://www.sparkfun.com/hobby-motor-with-encoder-
metal-gear-dg01d-e.html
                    //and from asking on discord
double degreesPerTick = 360.0/ticksPerRev/20.0 ;//20 for the gear reduction
double targetAngle; //in degrees

double value; //temp
void findAngle(double x, double y){
  Px += x; //what change were making
  Py += y;
  // Serial.print("PX: "); Serial.println(Px); //for testing reasons
  // Serial.print("Py: "); Serial.println(Py);

  BP = sqrt(sq(Px)+sq(Py)); //check photo for logic
  AP = sqrt(sq(Px - Ax) + sq(Py - Ay));

  gamma = atan2(Py, Px) * 180/PI; //radians to degrees
  value = (sq(CP)-sq(BC)-sq(BP))/(-2*BC*BP); //degrees
    if(value < -1) value = -1; //to make sure we get real values/ dont get nan
    if(value > 1) value = 1;
  delta = acos(value) * 180/PI;
  thetaB = 180 - gamma - delta; //servo takes angle from the other directions so
we need supplement angle

  beta = atan2(Py - Ay, Px - Ax) * 180 / PI; //degrees
  value = (sq(DP)-sq(AD)-sq(AP))/(-2*(AP)*(AD)); //degrees
    if(value < -1) value = -1; //check
    if(value > 1) value = 1;
  alpha = acos(value) * 180/PI;
  thetaA = 180 - beta - alpha; //motor takes angle from the other directions so
we need supplement angle

  // Physical limits to not break things
  thetaB = constrain(thetaB, 45, 90);
  thetaA = constrain(thetaA, 0, 90);
```

```
  //test Prints (can ignore for finished program)
  // Serial.print("thetaB ");Serial.println(thetaB);
  // Serial.print("delta ");Serial.println(delta);
  // Serial.print("gamma ");Serial.println(gamma);
  // Serial.print("beta ");Serial.println(beta);
  // Serial.print("alpha ");Serial.println(alpha);
  // Serial.print("thetaA ");Serial.println(thetaA);
  blackServoPWM = map(thetaB, 45, 90, 1500, 1800);
  // Serial.print("blackservo ");Serial.println(blackServoPWM); //for testing
reasons
  servo2.writeMicroseconds(blackServoPWM);
  targetAngle = thetaA; //change target angle

}

double error = 1.5; //angle error so we dont get constant gittering to reach a
specific angle
void blueMotorToAngle(double angle) {
  double currentAngle = -blueMotor.getPosition() * degreesPerTick; //get degrees
ticks * degrees/ticks = degrees we have gone
  //^ negative because our "forward" is actually backwards forthe blue motor
  if (currentAngle < angle - error) { //if the new angle is greater than current
    currentAngle = -blueMotor.getPosition() * degreesPerTick;
   if (currentAngle < (angle - error)) { //move up until angle reached
      currentAngle = -blueMotor.getPosition() * degreesPerTick;

      Serial.print("CurrentAngle:"); //testing
      Serial.println(currentAngle);

      blueMotor.setEffort(400); //move motor
      delay(50); //need a delay so motor actually gets enough power to move
    }
  }
  else if (currentAngle > angle + error) { //if angle we need to go to is less
than current angle
    // Move backward (decrease angle)
    currentAngle = -blueMotor.getPosition() * degreesPerTick; /
    if (currentAngle > (angle + error)) { //move down
      currentAngle = -blueMotor.getPosition() * degreesPerTick;

      Serial.print("CurrentAngle:"); //testing
      Serial.println(currentAngle);

      blueMotor.setEffort(-400); //move motor
      delay(50);
```

```
    }
  }


  // Stop motor after reaching the target
  blueMotor.setEffort(0);
  currentAngle = -blueMotor.getPosition() * degreesPerTick;

}

//for romi nonblocking movement
void moveRobot(int distance, bool x, bool y, bool z) { //distance in cm
  startTime = millis(); //reset start time we call, the starting time for how
long we move the motor
  timeToRun = distance/speed * 1000; //
  CW = x; //directions booleans
  CCW = y;
  Backwards = z;
  isMoving = true;

}

void setup() {
  Serial.begin(9600);
  delay(500); // Pause to allow internal operations

  chassis.init();
  blueMotor.setup();
  blueMotor.reset();

  // Servo Setup
  servo1.attach();
  servo1.setMinMaxMicroseconds(1000, 2000);
  servo2.attach();
  servo1.setMinMaxMicroseconds(500, 2500); //from spec sheet
  servo1.writeMicroseconds(1490); //for some reason 1500 just slowly closes our
claw. Maybe we have the wrong PW ?
  servo2.writeMicroseconds(1800); //90 degree position (vertical) for black servo

  delay(500); // Pause to ensure all previous processes have completed
  Serial.println("Setup Completed");
}


void loop() {
```

```
  static uint8_t lastDisplayTime;
  if (isMoving) { //moves romi
        if (millis() - startTime < timeToRun) { //if calculated timeToRun is
greater then the amount of time we need to run (so current time - prevtime)
            //in the library setWheelSpeeds uses PID to ensures the wheels move
at the same speed (so therefore distance)
            //it also already knows the wheel circum. and other info so we dont
have to do any of those calculations! YAY!
            if(Backwards){
              chassis.setWheelSpeeds(speed, speed); //claw is backwards so
backwards is positive speed
            }
            else if(CW){
              chassis.setWheelSpeeds(speed, -speed); //CW
            }
            else if(CCW){
              chassis.setWheelSpeeds(-speed, speed); //CCW
            }
            else {
              chassis.setWheelSpeeds(-speed, -speed); // move forwards
            }
        }
        else {
            chassis.setWheelSpeeds(0, 0); // Stop after time is up
            isMoving = false; // Reset movement state
            CW = false;
            CCW = false;
            Backwards = false;
        }
  }

  blueMotorToAngle(targetAngle); //update blue motor functions call

  // Keyboard controls
  if (Serial.available()) { // If a command is waiting
    char input = Serial.read();

    if (input == 'w') { // Move forward 5 cm
      Serial.println("forward");
      moveRobot(5, false, false, false);
    }
    else if (input == 'a') { // Turn left 13 cm (turns 90 degrees, should be
about 12 but that undershoots it)
      Serial.println("CCW");
      moveRobot(13, false, true, false);
```

```
    }
    else if (input == 's') {
      Serial.println("base backwards"); //backwards 5 cm
      moveRobot(5, false, false, true);
    }
    else if (input == 'd') {
      Serial.println("CW"); //turn right 13 cm
      moveRobot(13, true, false, false);
    }
    else if (input == 'o') { // claw opens
      Serial.println("open claw");
      servo1.writeMicroseconds(1300);
      delay(500);
      servo1.writeMicroseconds(1490); //for some reason 1500 makes it slowly
close so were using 1490 for stop moving
      //idk why lol
    }
    else if (input == 'c') { // close claw position
      Serial.println("close claw");
      servo1.writeMicroseconds(1700); //runs to keep closing to ensure could
contact/tension to lift load
    }
    else if (input == 'f'){
      //move five bar forward by xchange amount (mm)
      Serial.println("forward");
      findAngle(xchange, 0);

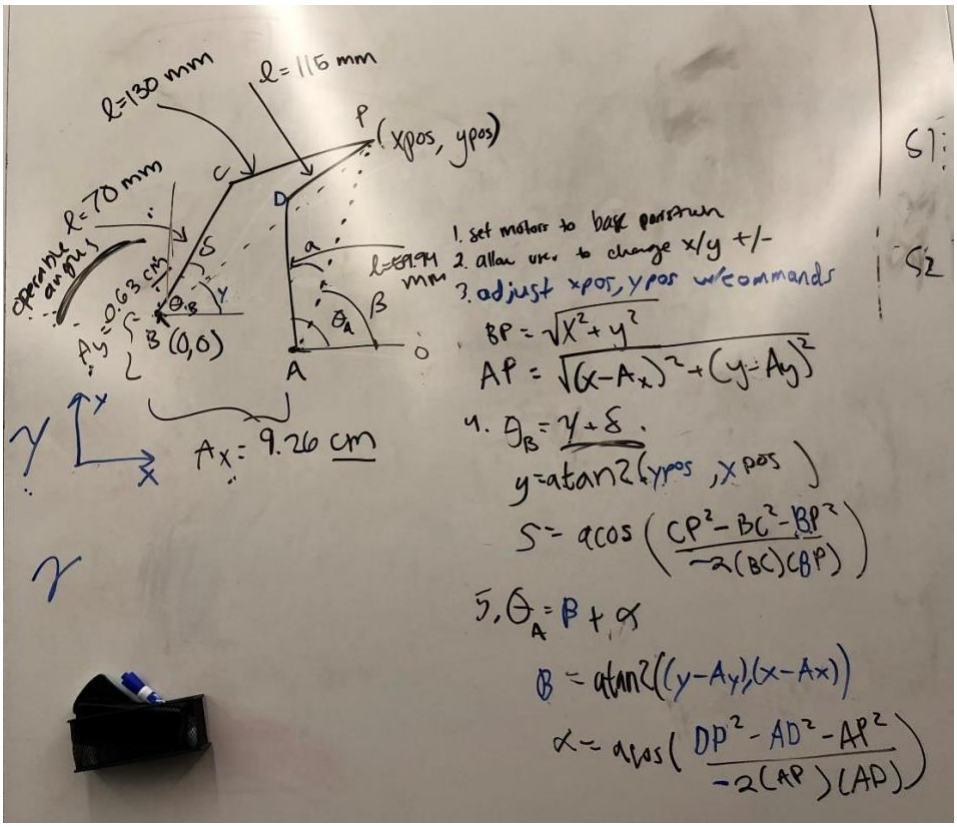
    }
    else if (input == 'b'){
      //move 5 bar back
      Serial.println("back");
      findAngle(-xchange,0);

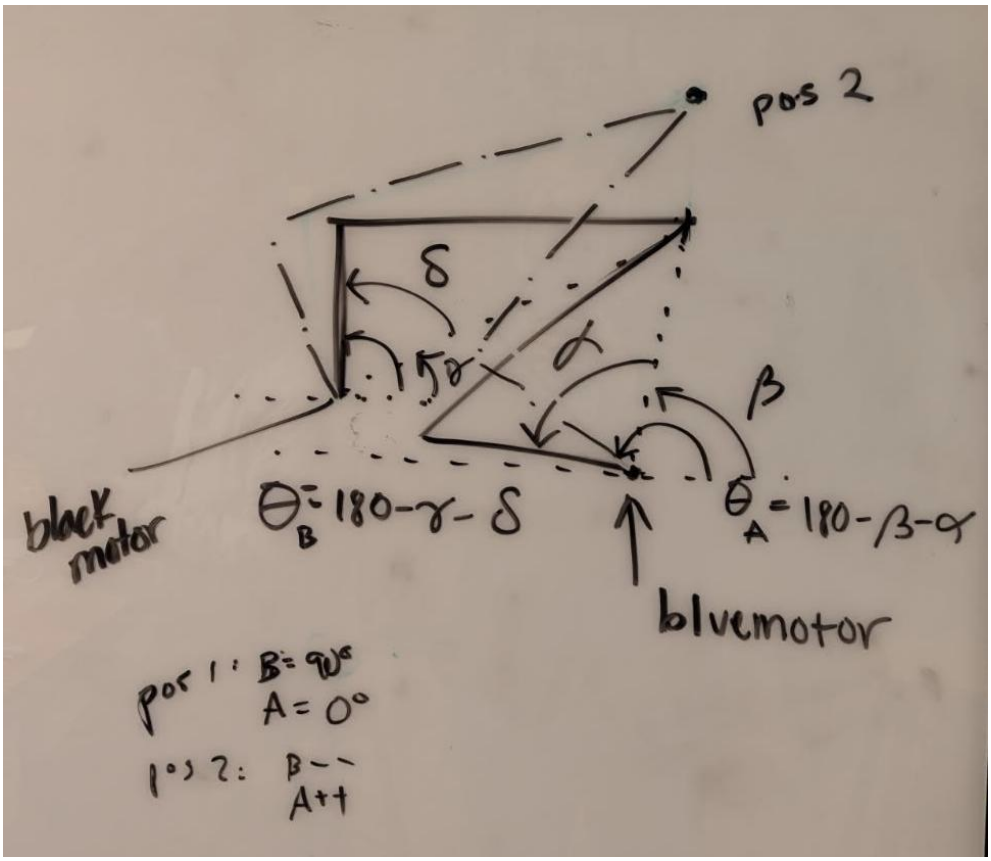
    }
    else if (input == 'u'){
      //move 5 bar up
      Serial.println("up");
      findAngle(0, ychange);

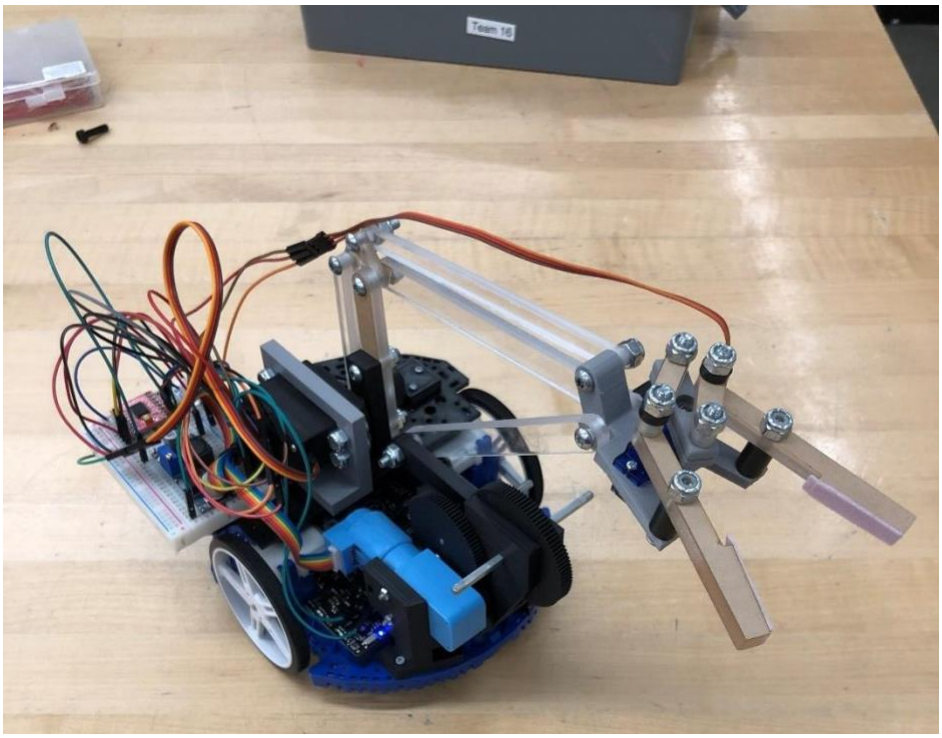
    }
    else if (input == 'l'){
      Serial.println("lower");
      findAngle(0, -ychange);
      //move 5 bar down
    }
```

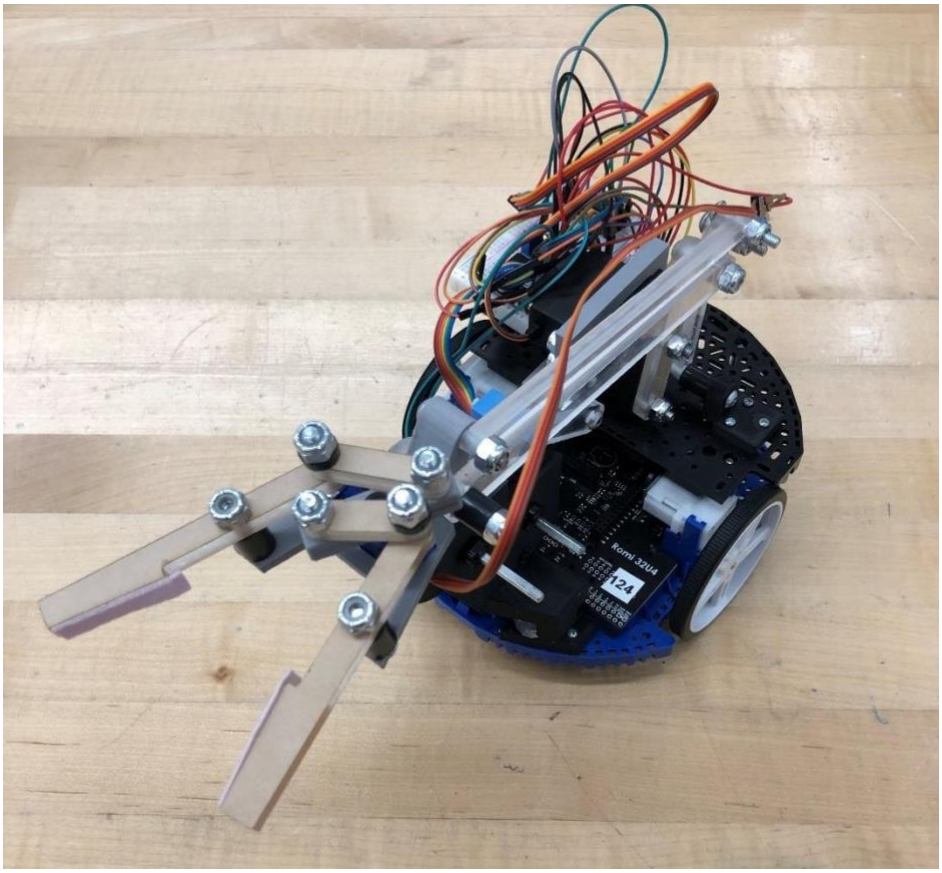
```
    else if (input == 'r') { // reset encoder and starting position
      blueMotor.reset();
       Px = 130;
       Py = 50;
       xchange = 0;
       ychange = 0;
    }
  }
}
```

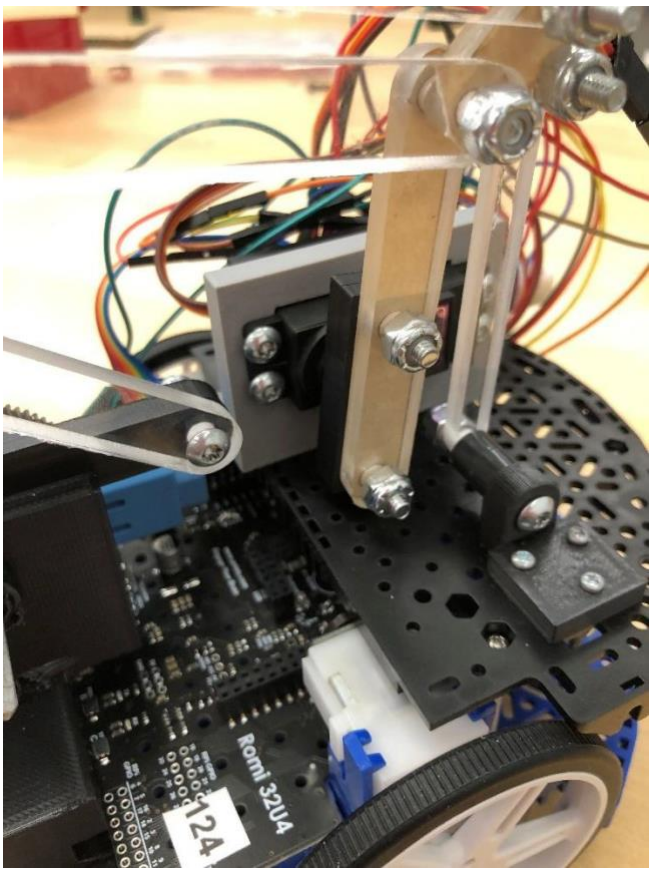
Extra Photos:




pos 2
δ
γ
α
β
black motor
$\theta_B = 180 - \gamma - \delta$
$\theta_A = 180 - \beta - \alpha$
blue motor
pos 1: B = 90°
A = 0°
pos 2: B ~
A ↑


Team 16


Romi 32U4
124


Romi 32U4
124

